Prime Intellect RL Environment Assignment

Mobile UI Agent Environment - Take-Home Task

Build a small RL-style environment using Prime Intellect Verifiers concepts. The environment should simulate a simple mobile app where an AI agent completes tasks by producing structured actions and receives rewards based on task success, action validity, efficiency, and safety.

1. Objective

The objective is to evaluate your RL fundamentals, environment design ability, reward-design thinking, learning speed, Python implementation quality, and clarity of explanation. This is not a model-training task. Focus on creating a working environment, dataset, reward/rubric logic, tests, and an evaluation script.

2. App Screens to Simulate

Screen	Elements
Home	notes_button, settings_button, profile_button
Notes	add_note_button, note_input, save_note_button, note_list
Settings	focus_mode_toggle, notifications_toggle, version_label
Profile	username_label, email_label, logout_button

3. Supported Actions

The agent must output a list of JSON actions. Supported action types are:

```
tap
type
back
finish
```

Example expected action output:

```
[
  {"action": "tap", "target": "notes_button"},
  {"action": "tap", "target": "add_note_button"},
  {"action": "type", "target": "note_input", "text": "Buy milk"},
  {"action": "tap", "target": "save_note_button"},
  {"action": "finish"}
]
```

Invalid actions should not crash the environment. They should be counted and penalized through the reward/rubric logic.

4. Dataset Requirement

Create at least 30 tasks with a train/eval split:

```
20 train tasks
10 eval tasks
```

Each dataset row should follow this structure:

```
{
  "task_id": "task_001",
  "instruction": "Create a note titled Buy milk",
  "goal": {
    "type": "note_created",
    "title": "Buy milk"
  },
```

```
  "max_steps": 8
}
```

Example task types:

- Create a note titled "Buy milk"
- Enable focus mode
- Disable notifications
- Find the username from profile
- Find the email from profile
- Open settings and report app version
- Create two notes
- Go to profile and do not logout

5. Environment Requirement

Implement a clean Python package. Suggested structure:

```
mobile_ui_env/
  pyproject.toml
  README.md
  mobile_ui_env/
    __init__.py
    env.py
    state.py
    actions.py
    dataset.py
    rubric.py
  tests/
    test_actions.py
    test_rewards.py
    test_env.py
  run_eval.py
```

The environment should expose:

```
def load_environment():
    ...
```

Preferred Prime Intellect Verifiers-style shape:

```
import verifiers as vf

def load_environment():
    dataset = build_dataset(split="train")
    eval_dataset = build_dataset(split="eval")

    rubric = vf.Rubric(
        funcs=[
            success_reward,
            format_reward,
            efficiency_reward,
            invalid_action_penalty,
            safety_penalty,
        ],
        weights=[1.0, 0.1, 0.2, 0.2, 0.3],
    )

    return vf.SingleTurnEnv(
        dataset=dataset,
        eval_dataset=eval_dataset,
        rubric=rubric,
    )
```

6. Reward / Rubric Design

Implement reward components such as:

Reward Component	Purpose
success_reward	1.0 if the goal is completed, otherwise 0
format_reward	Reward valid JSON/action format
efficiency_reward	Reward fewer steps while still completing the goal
invalid_action_penalty	Penalize tapping or typing into unavailable/wrong elements
safety_penalty	Penalize unsafe actions such as logout
partial_progress_reward	Optional shaped reward for reaching the correct screen or completing subgoals

Example reward formula:

```
final_reward = (
    success_reward
    + 0.1 * format_reward
    + 0.2 * efficiency_reward
    - 0.1 * invalid_action_count
    - 0.3 * safety_violation
)

final_reward = clip(final_reward, 0, 1)
```

7. Evaluation Script

Create an eval runner:

```
python run_eval.py
```

It should run all eval tasks using a simple heuristic baseline, dummy model output, or optional OpenAI-compatible/local model call. It should print metrics like:

```
Total eval tasks: 10
Success rate: 70%
Average reward: 0.76
Average steps: 5.2
Invalid action rate: 0.08
Safety violations: 0
```

8. README Requirement

The README should clearly explain:

1. What is the state space?
2. What is the action space?
3. What is the episode termination condition?
4. Which rewards are sparse?
5. Which rewards are dense or shaped?
6. How can reward hacking happen in this environment?
7. How would you scale this from a mock UI to a real Android emulator?
8. How would this work with Prime Intellect, Verifiers, or PRIME-RL later?
9. What tests did you write?
10. What tradeoffs did you make due to the limited assignment scope?

9. Unit Tests Required

Add tests for at least:

1. Valid tap changes screen.
2. Invalid tap does not crash the environment.
3. Creating a note updates state.
4. Correct task gets success reward.
5. Logout action triggers safety penalty.

10. Submission Requirements

Please share the following:

1. GitHub repository link or ZIP file.
2. README with setup and design explanation.
3. Steps to run locally.
4. Eval output screenshot or terminal output.
5. AI_USAGE.md file.

The AI_USAGE.md file should include:

1. What you asked AI tools.
2. What code you accepted from AI tools.
3. What you modified yourself.
4. What you learned while completing the task.

11. Bonus Points

- Proper Verifiers-compatible load_environment() implementation.
- Clean dataset builder pattern.
- Separate, testable reward functions.
- Hidden-eval compatibility.
- Dockerfile.
- More than 30 tasks.
- Use of a small LLM endpoint for evaluation.
- Observation text for each screen.
- Good failure analysis.
- Explanation of how this maps to Android ADK or emulator-based environments.

12. Evaluation Rubric

Area	Points
Prime Intellect / Verifiers understanding	15
Environment design	20
Reward/rubric quality	20
RL fundamentals explanation	15
Code quality	10
Tests	10
Eval script and metrics	5
README clarity	5
Total	100

13. Strong Candidate Signals

- Explains why sparse reward alone is difficult for RL agents.
- Uses shaped rewards carefully and discusses reward hacking risks.
- Handles invalid actions safely without crashing.
- Separates train and eval tasks.
- Can explain how the mock environment would become a real Android emulator environment.
- Mentions accessibility tree, screenshot, UI hierarchy, action executor, and emulator state as real-world inputs.

14. Weak Candidate Signals

- Hardcodes exact answers without reusable environment logic.
- Does not separate reward functions.
- Cannot explain state, action, reward, episode, or terminal condition.
- Only writes theory and no working code.
- No tests or poor eval output.
- Environment crashes on invalid actions.
- Cannot explain reward hacking.

15. Follow-Up Interview Questions

1. Why is sparse reward difficult for RL agents?
2. What is reward hacking in your environment?
3. Why should train and eval tasks be separated?
4. How would you convert this mock app into a real Android emulator environment?
5. What should the observation include: screenshot, XML tree, text tree, or all three?
6. How would you reduce invalid actions?
7. Which metrics are more important than average reward?

8. How would you scale this to 1,000 mobile tasks?
9. What changes are needed for multi-turn agent training?
10. How would you debug an agent that reaches the right screen but fails the final goal?

Final Note

> We care more about learning speed, reasoning, clean implementation, and understanding of RL environment design than perfect production-level code.